

Universidade Federal da Bahia
Instituto de Computação

Tarefa Avaliativa N1 - Representação de conhecimento em Prolog

MATA64 - Inteligência Artificial

Aluno: Matheus Pedreira dos Santos França
Professor: Luciano Rebouças de Oliveira

2026.2

1 Domínio Escolhido

Este projeto representa, em Prolog, as relações de construção entre um conjunto selecionado de itens de *League of Legends*. A base classifica os itens como básicos, intermediários ou finais e registra quais componentes entram diretamente na receita de cada item. A partir dessas relações, é possível identificar componentes indiretos, descobrir de quais itens finais um componente participa e comparar árvores de construção para encontrar componentes compartilhados ou exclusivos. O modelo registra a presença de cada componente, sem representar quantidades necessárias.

2 Base de Conhecimento

A base de conhecimento deste projeto inclui 23 fatos e 12 cláusulas de regra. Entre os fatos, o predicado `direct_component(C, I)` declara que `C` é um componente direto da receita de `I`. Essa orientação dos argumentos permite consultar tanto os componentes de um item quanto os itens que utilizam determinado componente.

As regras derivam informações não declaradas diretamente. O predicado `component_of/2` tem um caso base, que reconhece componentes diretos, e um caso recursivo, que percorre as etapas intermediárias da construção. Desse modo, a base pode inferir que um item básico participa da construção de um item final mesmo sem existir um fato que ligue diretamente os dois. Outras regras restringem consultas aos itens finais e comparam suas árvores de construção. Nessa comparação, um componente "exclusivo"é aquele registrado na árvore de um item, mas não na do outro, considerando apenas as receitas presentes na base.

3 Consultas e Resultados

As consultas abaixo foram executadas no SWI-Prolog com a base de conhecimento descrita anteriormente e implementada no arquivo `lol_items_knowledge_base.pl`. Quando uma consulta admite mais de uma solução, utilizou-se `;` para solicitar as respostas seguintes. Os resultados abaixo são apresentados conforme a saída da execução.

Consulta 1

```
?- direct_component(X, ludens_echo).
X = lost_chapter ;
X = hextech_alternator.
```

Consulta 2

```
?- direct_component(X, Y).
X = amplifying_tome,
Y = lost_chapter ;
X = sapphire_crystal,
Y = lost_chapter ;
X = glowing_mote,
Y = lost_chapter ;
X = amplifying_tome,
Y = hextech_alternator ;
X = amplifying_tome,
Y = fated_ashes ;
X = lost_chapter,
Y = ludens_echo ;
X = hextech_alternator,
Y = ludens_echo ;
X = needlessly_large_rod,
Y = rabadons_deathcap ;
X = hextech_alternator,
Y = shadowflame ;
X = needlessly_large_rod,
Y = shadowflame ;
X = lost_chapter,
Y = blackfire_torch ;
X = fated_ashes,
Y = blackfire_torch.
```

Consulta 3

```
?- component_of(X, ludens_echo).
X = lost_chapter ;
X = hextech_alternator ;
X = amplifying_tome ;
X = sapphire_crystal ;
X = glowing_mote ;
X = amplifying_tome ;
false.
```

Consulta 4

```
?- final_item_uses(Item, amplifying_tome).
Item = ludens_echo ;
Item = ludens_echo ;
Item = shadowflame ;
Item = blackfire_torch ;
Item = blackfire_torch ;
false.
```

Consulta 5

```
?- crafted_intermediate(lost_chapter).
true ;
true ;
true.
```

Consulta 6

```
?- shared_component(ludens_echo,
    shadowflame, X).
X = hextech_alternator ;
X = amplifying_tome ;
X = amplifying_tome ;
false.
```

As respostas repetidas nas consultas 3, 4 e 6 correspondem a caminhos distintos de inferência, não a quantidades na receita. Na consulta 5, os três resultados `true` ocorrem porque a condição `direct_component(_,lost_chapter)` pode ser satisfeita por três fatos diferentes. Assim, o Prolog encontra três provas para a mesma consulta.

O backtracking é evidenciado, por exemplo, na Consulta 4. Após cada resposta, o uso de `;` solicita outra solução para `final_item_uses/2`, até que não restem alternativas.

O arquivo `lol_items_knowledge_base.pl`, as capturas da execução das consultas apresentadas neste relatório e de consultas adicionais, bem como as árvores de construção utilizadas, estão disponíveis no repositório do projeto: https://github.com/mpsfranca/ia-ufba-2026.2-pl.